

Python: Recap

This notebook was generated in **pseudocode** mode.

Exercise 1: Sum of Two Integers

Exercise Description

1. Define two variables a and b with initial values 10 and 12, respectively.
2. Define the variable c as the sum of a and b
3. Print c

Your solution

```
# assign value to variable  
# assign value to variable  
# perform calculation and store result  
# print the result
```

Test your solution

```
assert c == 22
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 2: Area of a Triangle

Exercise Description

1. Compute the area of a triangle with:
 - base: 5.0
 - height: 7.0
2. Store the result in the variable area
3. Print the area

Your solution

```
# assign value to variable  
# assign value to variable
```

```
# perform calculation and store result  
# print the result
```

Test your solution

```
assert area == 17.5
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 3: Volume of a Cube

Exercise Description

1. Compute the volume of a cube with side equal to 8.0
2. Store the result in the variable volume
3. Print the volume

Your solution

```
# assign value to variable  
# perform calculation and store result  
# print the result
```

Test your solution

```
assert volume == 512.0
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 4: Compute Equation

Exercise Description

1. Define:

- x equal to 10
- y equal to 20

2. Compute the result of: $((x-4)^3 + 5) / (4 * (y \bmod 3))$

3. Store the result in the variable result

4. Print the result

Your solution

```
# assign value to variable  
# assign value to variable  
# perform calculation and store result  
# print the result
```

Test your solution

```
assert result == 27.625
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 5: Count Characters

Exercise Description

1. Define the string `s` equal to `Bazinga!`
2. Count the number of characters in `s` (without using a loop!) and store the result in the variable `length`
3. Print `length`

Your solution

```
# assign value to variable  
# calculate length and store in variable  
# print the result
```

Test your solution

```
assert length == 8
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 6: Concatenate Strings

Exercise Description

1. Define one string `s1` with the content `Francesco`
2. Define one string `s2` with the content (one space)
3. Define one string `s3` with the content `Totti`
4. Concatenate `s1`, `s2`, and `s3` into `s4`
5. Print the concatenated string `s4`

Your solution

```
# assign value to variable  
# assign value to variable  
# assign value to variable  
# perform calculation and store result  
# print the result
```

Test your solution

```
assert s4 == "Francesco Totti"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 7: Last Three Characters

Exercise Description

1. Define the string `s1` with the content `Totti`
2. Extract the last three characters of `s1` into `s2`
3. Print the string `s2`

Your solution

```
# assign value to variable  
# perform calculation and store result  
# print the result
```

Test your solution

```
assert s2 == "tti"
```


Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 8: Convert Types

Exercise Description

1. Define the **integer** `n1` equal to 10
2. Define the **string** `s1` equal to "20.5"
3. Convert `n1` and `s1` to float and then sum the two values into `f1`
4. Print `f1`

Your solution

```
# assign value to variable  
# assign value to variable  
# perform calculation and store result  
# print the result
```

Test your solution

```
assert f1 == 30.5
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 9: Largest Average of Two Sequences

Exercise Description

1. Define the numbers `a1`, `a2`, and `a3` with values equal to 10, 12, 8, respectively
2. Define the numbers `b1`, `b2`, and `b3` with values equal to 7, 10, 18, respectively
3. Compute the average of:
 - `a1`, `a2`, and `a3` into `avg_a`
 - `b1`, `b2`, and `b3` into `avg_b`
4. Conditionally define the string `s` based on which average is larger
5. Print `avg_a`, `avg_b`, `s`

Your solution

```
# assign value to variable
# assign value to variable
# assign value to variable
# assign value to variable
# assign value to variable
# assign value to variable
# perform calculation and store result
# perform calculation and store result
# check condition
    # assign value to variable
# check alternative condition
    # assign value to variable
# handle remaining cases
    # assign value to variable
# print the result
# print the result
# print the result
```

Test your solution

```
assert round(avg_a, 1) == 10.0
```

Test your solution

```
assert round(avg_b, 1) == 11.7
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 10: Perfect Square

Exercise Description

1. Define the numbers `n1` and `n2` with values equal to 9 and 12, respectively
2. Check whether these numbers are *perfect squares*
3. Store the result of the checks in `is_n1_perfect_square` and `is_n2_perfect_square`
4. Print the results

Your solution

```
# assign value to variable
# assign value to variable
# check condition
    # assign value to variable
# handle remaining cases
    # assign value to variable
# check condition
    # assign value to variable
# handle remaining cases
```

```
# assign value to variable  
# print the result  
# print the result
```

Test your solution

```
assert is_n1_perfect_square == True
```

Test your solution

```
assert is_n2_perfect_square == False
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 11: Count Digits

Exercise Description

1. Define the integer n1 equal to 123450
2. Using a loop, count the number of digits in n1 and store the result in ndigits
3. Print ndigits

Your solution

```
# assign value to variable  
# assign value to variable  
# repeat while condition is true  
    # perform calculation and store result  
    # perform calculation and store result  
# print the result
```

Test your solution

```
assert ndigits == 6
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 12: Quotient and Remainder by Hand

Exercise Description

1. Define the integer `n1` equal to 123450
2. Define the integer `n2` equal to 57
3. Using a loop, without using the `/` and/or `//` and/or `%` operators, compute the quotient `q` and the remainder `r` of the integer division of `n1` by `n2`
4. Print `r` and `q`

Your solution

```
# assign value to variable n1  
# assign value to variable n2  
# assign value to variable r  
# assign value to variable q  
# repeat while r is greater than or equal to n2  
    # subtract n2 from r  
    # add 1 to q  
# print the value of q and r
```

Test your solution

```
assert q == 2165
```

Test your solution

```
assert r == 45
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 13: Sum of First N Numbers

Exercise Description

1. Define the integer n equal to 100
2. Using a loop compute the sum of the first n numbers (starting from 1), storing the result into s
3. Print s

Your solution

```
# assign value to variable  
# assign value to variable  
# repeat while condition is true  
# perform calculation and store result
```



```
# perform calculation and store result  
# print the result
```

Test your solution

```
assert s == 5050
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 14: Sum of Prime Numbers

Exercise Description

1. Define the integer n equal to 100
2. Using a loop compute the sum of **prime** numbers up to n , storing the result into s
3. Print s

Your solution

```
# assign value to variable
# assign value to variable
# iterate through sequence
    # check condition
        # assign value to variable
    # handle remaining cases
        # assign value to variable
        # iterate through sequence
            # check condition
                # assign value to variable
    # check condition
        # perform calculation and store result
# print the result
```

Test your solution

```
assert s == 1060
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 15: Max of a List

Exercise Description

Define a function `max_from_list` that:

- takes as arguments a list of integers
- returns:
 - if the list is not empty: the maximum value in the list
 - otherwise: `None`

Do not use the built-in function `max` in this exercise.

Your solution

```
def max_from_list(L):  
    # assign value to variable  
    # iterate through sequence  
        # check condition  
            # assign value to variable  
    # return the result
```

Test your solution

```
assert max_from_list([]) == None
```

Test your solution

```
assert max_from_list([1, 2, 3]) == 3
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 16: find number of unique characters

Exercise Description

find number of unique characters

Define a function `count_uniq` that:

- takes as arguments:
 - a string `s`
- returns:
 - the number of unique characters in `s`

Your solution

```
def count_uniq(s):  
    # step 1: create a set from the string  
    # step 2: return the length of the set  
    pass
```

Test your solution

```
assert count_uniq("test") == 3
```

Test your solution

```
assert count_uniq("Aejeje") == 3
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 17: remove duplicates

Exercise Description

remove duplicates

Define a function `remove_duplicates` that:

- takes as arguments:
 - a list `s` of strings
- returns:
 - a copy of `s` without duplicate elements

Your solution

```
def remove_duplicates(s):  
    # step 1: create a set from the list  
    # step 2: convert the set back to a list  
    # step 3: return the list  
    pass
```

Test your solution

```
assert sorted(remove_duplicates(["test", "luiss", "data", "test", "science"])) == sorted(["test", "luiss", "data", "science"])
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code**

before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 18: find common elements

Exercise Description

find common elements

Define a function `common_elements` that:

- takes as arguments:
 - a set `s` of strings
 - a set `k` of strings
- returns:
 - a set containing all common elements between `s` and `k`

Your solution

```
def common_elements(s, k):  
    # step 1: return the intersection of the two sets  
    pass
```

Test your solution

```
friend1_companies = {'Google', 'Amazon', 'Apple', 'Microsoft'}  
friend2_companies = {'Facebook', 'Google', 'Tesla', 'Amazon'}  
try: assert common_elements(friend1_companies, friend2_companies) == {'Google', 'Amazon'}  
except: print('Test failed')
```

Test passed

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 19: count word frequency

Exercise Description

count word frequency

Define a function `word_freq` that:

- takes as arguments:
 - a string `s`
- returns:
 - a dictionary containing the frequency (value) within `s` of each word (key) contained in `s`.

NOTE: - count the word case-insensitive. - split ``s`` by space to obtain the words.

Your solution

```
def word_freq(s):  
    # step 1: Convert the input string 's' to lowercase  
    # step 2: Split the lowercase string into individual words  
    # step 3: Create an empty dictionary to store word frequencies  
    # step 4: Iterate over each word in the list of words  
        # step 5: Increment the word's count in the dictionary by 1  
        # step 6: If the word is not already in the dictionary, set its count to 1  
    # step 7: Return the dictionary containing word frequencies  
    pass
```

Test your solution

```
try: assert word_freq("Python is fun and learning Python is fun") == {'python': 2, 'i  
except: print('Test failed')
```

Test passed

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 20: track voting results

Exercise Description

track voting results

Define a function `update_votes` that:

- takes as arguments:
 - a dictionary `votes` having as key names of candidates and as value the number of votes received by each one of them
 - a list `new_votes` of names of candidates
- returns:
 - the `votes` dictionary updated with the new votes received

Your solution

```
def update_votes(votes, new_votes):  
    # step 1: iterate over each vote in new_votes  
    # step 2: update the count of each vote in votes  
    # step 3: return the updated votes  
    pass
```

Test your solution

```
votes = {  
    'Alice': 120,  
    'Bob': 150,  
    'Charlie': 90  
}  
  
new_votes = ['Alice', 'Charlie', 'Charlie', 'Bob', 'Alice', 'Alice']  
try: assert update_votes(votes, new_votes) == {'Alice': 123, 'Bob': 151, 'Charlie': 92}  
except: print('Test failed')
```

Test passed

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 21: Greatest Common Divisor (GCD)

Exercise Description

Greatest Common Divisor (GCD)

Define a function gcd that:

- takes as argument two integer numbers a and b
- returns:
 - the gcd between a and b

Your solution

```
def gcd(x, y):  
    # step 1: initialize a variable to store the gcd  
    # step 2: while y is not 0  
    #     - update x to y  
    #     - update y to the remainder of x divided by y  
    # step 3: return x  
    pass
```

Test your solution

```
assert gcd(1,2) == 1  
assert gcd(7,2) == 1
```

```
assert gcd(4,2) == 2
assert gcd(15,25) == 5
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 22: Longest Substring Without Repeating Characters

Exercise Description

Longest Substring Without Repeating Characters

Define a function `longest_unique_substring` that:

- Takes a string as input.
- Returns the length of the longest substring that contains only unique characters.

Example:

- `longest_unique_substring("abcbcb")` should return 3 (substring "abc").
- `longest_unique_substring("bbbb")` should return 1.

Your solution

```
def longest_unique_substring(s):  
    # step 1: initialize zero to max_length variable to store the length of the longest unique substring  
    # step 2: iterate over the indices of the string s  
        # step 3: initialize an empty set to keep track of characters in the current substring  
        # step 4: iterate over the indices of the string s starting from the current index  
            # step 5: if the current character is already in the set, break the loop  
            # step 6: add the current character to the set  
            # step 7: update max_length to be the maximum of max_length and the length of the current substring  
    # step 8: return max_length  
    pass
```

Test your solution

```
assert longest_unique_substring("abcbcb") == 3  
assert longest_unique_substring("bbbb") == 1
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 23: Find the Majority Element

Exercise Description

Find the Majority Element

Define a function `majority_element` that:

- Takes a list of integers as input.
- Returns the element that appears more than half of the time in the list (if it exists). If no such element exists, return `None`.

Example:

- `majority_element([3, 3, 4, 2, 3, 3, 5])` should return 3.
- `majority_element([1, 2, 3, 4, 5])` should return `None`.

Your solution

```
def majority_element(nums):  
    # step 1: Initialize an empty dictionary 'count' to store the frequency of each number  
    # step 2: Get the length of the input list 'nums'  
    # step 3: Iterate over each number in the list 'nums'  
        # step 4: If the number 'num' is already in the count dictionary, increment its value  
        # step 5: If the number 'num' is not in the count dictionary, add it with a value of 1
```

```
# step 6: Check if the count of the current number exceeds half of the list  
# step 7: If so, return the current number as the majority element  
# step 8: If no majority element is found, return None  
pass
```

Test your solution

```
assert majority_element([3, 3, 4, 2, 3, 3, 5]) == 3
```

Test your solution

```
assert majority_element([1, 2, 3, 4, 5]) == None
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 24: Implement ROT13 Cipher

Exercise Description

Implement ROT13 Cipher

Define a function `rot13` that:

- Takes a string as input.
- Returns a new string where each letter is replaced by the letter 13 positions after it in the alphabet. If the shift passes the end of the alphabet, it wraps around to the beginning. Non-alphabetic characters should remain unchanged.

NOTE: - ROT13 is a special case of the Caesar cipher, which is a simple substitution cipher where each letter in the plaintext is shifted a certain number of places down or up the alphabet. In the case of ROT13, the shift is 13 places. - Consider an alphabet of 26 letters: ``"abcdefghijklmnopqrstuvwxyz"`` - ``ord(c)`` returns an integer representing ``c``. - ``chr(x)`` returns the character associated with integer ``x``.

Example:

- `rot13("hello")` should return `"uryyb"`.
- `rot13("uryyb")` should return `"hello"`.
- `rot13("hello, world!")` should return `"uryyb, jbeyq!"`.

Your solution

```
def rot13(s):  
    # Step 1: Initialize an empty list 'result' to store the transformed characters  
    # Step 2: Iterate over each character in the input string 's'  
    # Step 3: Check if the character is a lowercase letter
```

```
# Step 4: Apply ROT13 transformation: shift character by 13 positions with  
# Step 5: Check if the character is an uppercase letter  
# Step 6: Apply ROT13 transformation: shift character by 13 positions with  
# Step 7: If the character is neither lowercase nor uppercase, keep it unchanged  
# Step 8: Join the list of transformed characters into a single string and return it  
pass
```

Test your solution

```
assert rot13("hello") == "uryyb"  
assert rot13("uryyb") == "hello"  
assert rot13("hello, world!") == "uryyb, jbeyq!"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 25: Multiply Tuples

Exercise Description

Multiply Tuples

Write a function called `mul_tuple` that:

- Takes two tuples containing integers as arguments
- Returns a new tuple containing the products of the corresponding elements (at the same position) of the two tuples. If the two tuples have different lengths, the function should return `None`.

Your solution

```
# Define function mul_tuple that takes parameter tuple1, tuple2  
# If len(tuple1) != len(tuple2):  
    # Return None (no result)  
# Set res to tuple()  
# For each i in range(len(tuple1)):  
    # Set res to res + (tuple1[i] * tuple2[i], )  
# Return the result: res
```

Test your solution

```
assert mul_tuple((1, 2, 3), (4, 5, 6, 7)) == None  
assert mul_tuple((1, 2), (4, 5)) == (4, 10)
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 26: Max Point Distance

Exercise Description

Max Point Distance

Write a function `max_dist_point` that:

- Takes as arguments:
 - A point in the Cartesian plane represented as a tuple with its coordinates (x, y), where x and y are integers.
 - A list of points in the Cartesian plane.
- Returns:
 - If the list received as an argument is empty: None.
 - Otherwise: a tuple of two values, consisting of:
 1. The maximum distance (integer) between the given point and all the points in the list. To calculate the distance between a pair of points ((x1, y1)) and ((x2, y2)), use the Euclidean distance formula:

$$distance = \sqrt{(x2 - x1)^2 + (y2 - y1)^2}$$

2. The point from the list that produced the maximum distance. Round the distance down using `int(distance)`.

NOTE: The square root can be calculated using ``math.sqrt()`` from the math library.

Your solution

```
# Import the math module
# Define function max_dist_point that takes parameter punto, lista_punti
# If not lista_punti: return Non:
# Set distanza_massima to -1
# Set punto_massimo to None
```

```
# For each punto_lista in lista_punti:  
# Set distanza to int(math.sqrt((punto_lista[0] - punto[0])**2 + (punto_lista  
# If the value is greater than the threshold:  
# Set distanza_massima to distanza  
# Set punto_massimo to punto_lista  
# Return the result: (distanza_massima, punto_massimo)
```

Test your solution

```
assert max_dist_point((0, 0), []) == None  
assert max_dist_point((0, 0), [(1, 1), (2, 2), (3, 3)]) == (4, (3, 3))  
assert max_dist_point((10, 12), [(1, 3), (4, 23), (-100, 0), (1, 1)])
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 27: Character Position Tracker

Exercise Description

Character Position Tracker

Write a function `track_char_positions` that:

- Takes a string as input.
- Returns a dictionary where:
 - The keys are the unique characters in the string.
 - The values are lists of positions (indices) where each character appears in the string.

NOTE: - The function should track both uppercase and lowercase characters as distinct. - Spaces and punctuation should also be tracked as characters.

Your solution

```
# Define function track_char_positions that takes parameter text  
# Create an empty dictionary to store the character positions  
# Initialize char_positions as an empty dictionary  
# Loop through the string and keep track of the index of each character  
# For each index and char in enumerate(text):  
    # If the character is not already a key in the dictionary, add it  
    # If the item is not already in the collection:  
        # Initialize char_positions[char] as an empty list  
    # Append the index of the character to the list  
    # char_positions[char].append(index)  
# Return the result: char_positions
```

Test your solution

```

text = "hello"
expected_result = {'h': [0], 'e': [1], 'l': [2, 3], 'o': [4]}
try: assert track_char_positions(text) == expected_result and not print("Test #1 passed")
except: print('Test #1 failed')

text = "banana"
expected_result = {'b': [0], 'a': [1, 3, 5], 'n': [2, 4]}
try: assert track_char_positions(text) == expected_result and not print("Test #2 passed")
except: print('Test #2 failed')

text = "Hi, there !"
expected_result = {
    'H': [0], 'i': [1], ',': [2], ' ': [3, 9], 't': [4], 'h': [5], 'e': [6, 8], 'r': [7]
}
try: assert track_char_positions(text) == expected_result and not print("Test #3 passed")
except: print('Test #3 failed')

```

```

Test #1 passed
Test #2 passed
Test #3 passed

```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```

from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth

```


Exercise 28: Anagram Grouping

Exercise Description

Anagram Grouping

Write a function `group_anagrams` that:

- Takes a list of strings as input.
- Returns a dictionary where:
 - The keys are the strings received as input where their characters are sorted alphabetically.
 - The values are alphabetically sorted lists of words from the input list that are anagrams of each other.

NOTE: - The words should be grouped based on their sorted letter order. - If no anagram pairs are found, each word should still appear in its own list.

Your solution

```
# Define function group_anagrams that takes parameter words  
# Initialize anagram_dict as an empty dictionary  
# For each word in words:  
    # Sort the word to create a key  
    # Sort the items and assign to sorted_word
```

```

    # Add the word to the list corresponding to the sorted key
    # If the item is not already in the collection:
        # Initialize anagram_dict[sorted_word] as an empty list
    # anagram_dict[sorted_word].append(word)
# Sort the list of anagrams for each key
# For each each in anagram_dict:
    # Sort the items and assign to anagram_dict[each]
# Return the result: anagram_dict

```

Test your solution

```

words = ["listen", "silent", "enlist", "hello"]
expected_result = {
    'eilnst': ['enlist', 'listen', 'silent'],
    'ehllo': ['hello']
}
try: assert group_anagrams(words) == expected_result and not print("Test #1 passed")
except: print('Test #1 failed')

words = ["apple", "banana", "orange"]
expected_result = {
    'aelpp': ['apple'], 'aaabnn': ['banana'], 'aegnor': ['orange']
}
try: assert group_anagrams(words) == expected_result and not print("Test #2 passed")
except: print('Test #2 failed')

words = ["Listen", "Silent", "enlist"]
expected_result = {'Leinst': ['Listen'], 'Seilnt': ['Silent'], 'eilnst': ['enlist']}
try: assert group_anagrams(words) == expected_result and not print("Test #3 passed")
except: print('Test #3 failed')

```

Test #1 passed
Test #2 passed
Test #3 passed

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 29: ISBN Validator

Exercise Description

ISBN Validator

Write a function `validate_isbn` that:

- Takes a string `isbn` as input, representing a 10-digit ISBN number.
- Returns a dictionary containing:
 - `valid` (key): as value, a boolean indicating whether the ISBN is valid.
 - `digits` (key): as value, a list of the individual digits in the ISBN.

An ISBN is considered valid if it meets the following criteria:

1. It consists of exactly 10 characters (excluding hyphens or spaces, which are ignored) with an 'X' (which represents the number 10).
2. The ISBN is valid if the weighted sum of the digits (where the weight decreases from 10) calculation would be:
$$(0 \times 10) + (3 \times 9) + (0 \times 8) + (6 \times 7) + (4 \times 6) + (0 \times 5) + (6 \times 4) + (1 \times 3) + (5$$

Since $(132 \bmod 11 = 0)$, it is valid.

Your solution

```
# Define function validate_isbn that takes parameter isbn
# Remove any hyphens or spaces from the input
# Set isbn to isbn.replace("-", "").replace(" ", "")
# Check if the length is 10
# If len(isbn) != 10: return {'valid': False, 'digits': []}
# Initialize variables
# Set total to 0
# Initialize digits as an empty list
# For each i and char in enumerate(isbn):
    # If char.isdigit():
        # Set digit to int(char)
        # Add digit * (10 - i) # Weighted sum to total
        # Add char to the digits list
    # Otherwise, if char == 'X' and i == 9:
        # Add 10 # Last character can be 'X' to total
```

```
        # Add 'X' to the digits list
    # Otherwise:
        # Return the result: {'valid': False, 'digits': []} # Invalid character
# Check divisibility by 11
# valid = (total % 11 == 0)
# Return the result: {'valid': valid, 'digits': digits}
```

Test your solution

```
isbn = "0-306-40615-2"
expected_result = {'valid': True, 'digits': ['0', '3', '0', '6', '4', '0', '6', '1',
try: assert validate_isbn(isbn) == expected_result and not print("Test #1 passed")
except: print('Test #1 failed')

isbn = "123456789X"
expected_result = {'valid': True, 'digits': ['1', '2', '3', '4', '5', '6', '7', '8',
try: assert validate_isbn(isbn) == expected_result and not print("Test #2 passed")
except: print('Test #2 failed')

isbn = "12345678"
expected_result = {'valid': False, 'digits': []}
try: assert validate_isbn(isbn) == expected_result and not print("Test #3 passed")
except: print('Test #3 failed')
```

Test #1 passed
Test #2 passed
Test #3 passed

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 30: Acronym Generator

Exercise Description

Acronym Generator

Write a function `generate_acronym` that:

- Takes a string as input, representing a multi-word phrase (e.g., "As Soon As Possible").
- Returns a dictionary where:
 - The key is the acronym formed from the first letter of each word in the phrase (case insensitive).
 - The value is the original phrase with each word capitalized.

NOTE: - Ignore any non-alphabetic characters when forming the acronym. - The acronym should be in uppercase. - If the input string is empty, return `{'acronym': '', 'phrase': ''}`.

Your solution

```
# Define function generate_acronym that takes parameter s
# Split the string into words and assign to words
# Set key to ""
# Set value to ""
# For each word in words:
#   If the values are equal:
#     Skip to the next iteration
#   Set word to the lowercase version of word
#   Set wordUpper to word.upper()
#   Set c to wordUpper[0]
#   Set word to c + word[1:]
#   Add c to key
#   Add word + " " to value
# Set value to value.rstrip()
# Set r to {'acronym': key, 'phrase': value}
# Return the result: r
```

Test your solution

```
phrase = "as soon as possible"
expected_result = {'acronym': 'ASAP', 'phrase': 'As Soon As Possible'}
try: assert generate_acronym(phrase) == expected_result and not print("Test #1 passed")
except: print('Test #1 failed')

phrase = "    keep it simple stupid  "
expected_result = {'acronym': 'KISS', 'phrase': 'Keep It Simple Stupid'}
try: assert generate_acronym(phrase) == expected_result and not print("Test #2 passed")
except: print('Test #2 failed')

phrase = "for your information."
```

```
expected_result = {'acronym': 'FYI', 'phrase': 'For Your Information.'}
try: assert generate_acronym(phrase) == expected_result and not print("Test #3 passed")
except: print('Test #3 failed')
```

Test #1 passed
Test #2 passed
Test #3 passed

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 31: Movie Rating Organizer

Exercise Description

Movie Rating Organizer

Write a function `organize_movie_ratings` that:

- Takes a list of tuples as input, where each tuple contains two elements:
 - A string representing the name of a movie.
 - An integer representing the rating of that movie (from 1 to 10).
- Returns a dictionary where:
 - The keys are the unique movie titles.
 - The values are lists of ratings for each movie.

NOTE: - If a movie appears multiple times in the input list, all ratings should be included in the list for that movie. - The order of the ratings in the lists should reflect the order they appear in the input list.

Your solution

```
# Define function organize_movie_ratings that takes parameter ratings  
# Initialize movie_dict as an empty dictionary  
# For each movie and rating in ratings:  
    # If the item is not already in the collection:  
        # Set movie_dict[movie] to [] # Initialize the list for new movies  
        # movie_dict[movie].append(rating) # Append the rating to the list  
# Return the result: movie_dict
```

Test your solution

```
ratings = [  
    ("Inception", 9),  
    ("The Matrix", 8),  
    ("Inception", 10),
```

```

        ("The Godfather", 9),
        ("The Matrix", 9)
    ]
    expected_result = {
        'Inception': [9, 10],
        'The Matrix': [8, 9],
        'The Godfather': [9]
    }
    try: organize_movie_ratings(ratings) == expected_result and not print("Test #1 passed")
    except: print('Test #1 failed')

```

Test #1 passed

Test your solution

```

ratings = [
    ("Titanic", 7),
    ("Titanic", 7),
    ("Titanic", 7)
]
expected_result = {
    'Titanic': [7, 7, 7]
}
try: organize_movie_ratings(ratings) == expected_result and not print("Test #2 passed")
except: print('Test #2 failed')

ratings = [
    ("Avatar", 8),
    ("Avatar", 9),
    ("Avatar", 10)
]

```

```
expected_result = {
    'Avatar': [8, 9, 10]
}
try: organize_movie_ratings(ratings) == expected_result and not print("Test #3 passed")
except: print('Test #3 failed')
```

Test #2 passed
Test #3 passed

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 32: Contact Book

Exercise Description

Contact Book

Write a function `create_contact_book` that:

- Takes a list of tuples as input, where each tuple contains two elements:
 - A string representing the name of a contact.
 - A string representing the contact's phone number.
- Returns a dictionary where:
 - The keys are the unique names of the contacts (case insensitive).
 - The values are the corresponding phone numbers.

NOTE: - If a contact appears multiple times in the input list, the last occurrence should be kept in the dictionary. - The names in the dictionary should be in lowercase to maintain case insensitivity.

Your solution

```
# Define function create_contact_book that takes parameter contacts  
# Initialize contact_dict as an empty dictionary  
# For each name and phone_number in contacts:  
    # Use lowercase for names to ensure case insensitivity  
    # Set contact_dict[name.lower()] to phone_number  
# Return the result: contact_dict
```

Test your solution

```
contacts = [  
    ("Alice", "123-456-7890"),  
    ("Bob", "987-654-3210"),  
    ("alice", "555-555-5555"),  
    ("Charlie", "111-222-3333")  
]
```

```

]
expected_result = {
    'alice': '555-555-5555',
    'bob': '987-654-3210',
    'charlie': '111-222-3333'
}
try: assert create_contact_book(contacts) == expected_result and not print("Test #1 passed")
except: print('Test #1 failed')

```

Test #1 passed

Test your solution

```

contacts = [
    ("John", "555-123-4567"),
    ("john", "555-765-4321"),
    ("Doe", "555-987-6543")
]
expected_result = {
    'john': '555-765-4321',
    'doe': '555-987-6543'
}
try: assert create_contact_book(contacts) == expected_result and not print("Test #2 passed")
except: print('Test #2 failed')

# Test Case 3: Only one contact
contacts = [
    ("Alice", "123-456-7890")
]
expected_result = {
    'alice': '123-456-7890'
}

```

```
}  
try: assert create_contact_book(contacts) == expected_result and not print("Test #3 passed")  
except: print('Test #3 failed')
```

Test #2 passed

Test #3 passed

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 34: Social Media Connections

Exercise Description

Social Media Connections

Write a function `manage_connections` that:

- Takes a list of tuples as input, where each tuple contains:
 - A string representing the username.
 - A set of strings representing the usernames of friends that the user is connected to.
- The function should return a dictionary representing each user and their unique connections (friends) where:
 - The keys are unique usernames (case insensitive).
 - The values are sets of unique friends for that user.

NOTE: - If a user has multiple connections with the same friend, those should only be counted once. - The function should maintain case insensitivity for usernames (e.g., "Alice" and "alice" should be treated as the same user). - If a user has no friends, their value in the dictionary should be an empty set.

Your solution

```
# Define function manage_connections that takes parameter connections  
# Initialize user_connections as an empty dictionary  
# For each user and friends in connections:  
    # Normalize the username to lowercase for case insensitivity  
    # Set normalized_user to the lowercase version of user  
    # If the user is not in the dictionary, initialize their set  
    # If the item is not already in the collection:
```

```
# Initialize user_connections[normalized_user] as an empty set  
# Update the user's friends set with the provided friends  
# Update the collection with new items  
# Normalize friends to be case insensitive as well  
# For each user in user_connections:  
    # Set user_connections[user] to the lowercase version of {friend  
# Return the result: user_connections
```

Test your solution

```
connections = [  
    ("Alice", {"Bob", "Charlie"}),  
    ("Bob", {"Alice", "David"}),  
    ("alice", {"Eve"}),  
    ("Charlie", {"Bob"}),  
    ("david", {"Alice", "Eve"}),  
    ("Eve", set())  
]  
expected_result = {  
    'alice': {"bob", "charlie", "eve"},  
    'bob': {"alice", "david"},  
    'charlie': {"bob"},  
    'david': {"alice", "eve"},  
    'eve': set()  
}  
try: assert manage_connections(connections) == expected_result and not print("Test #1  
except: print('Test #1 failed')
```

Test #1 passed

Test your solution

```
connections = [
    ("John", set()),
    ("Doe", set())
]
expected_result = {
    'john': set(),
    'doe': set()
}
try: assert manage_connections(connections) == expected_result and not print("Test #2 passed")
except: print('Test #2 failed')

connections = [
    ("Alice", {"Bob", "Charlie"}),
    ("Alice", {"Bob", "Eve"}),
    ("bob", {"Alice"}),
    ("charlie", {"Alice"}),
]
expected_result = {
    'alice': {"bob", "charlie", "eve"},
    'bob': {"alice"},
    'charlie': {"alice"},
}
try: assert manage_connections(connections) == expected_result and not print("Test #3 passed")
except: print('Test #3 failed')
```

Test #2 passed

Test #3 passed

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 35: Company Employee Records

Exercise Description

Company Employee Records

Write a function `manage_employees` that:

- Takes a list of tuples as input, where each tuple contains:
 - A string representing the name of the department (e.g., "HR", "Engineering").
 - A string representing the name of the employee.
 - An integer representing the employee's salary (can be negative to indicate salary reductions).
- The function should return a dictionary representing each department's employees where:
 - The keys are unique department names (case insensitive).
 - The values are dictionaries containing:
 - `employees`: a dictionary of employee names (case insensitive) and their current salaries.
 - `average_salary`: the average salary of employees in that department, rounded to two decimal places.

Your solution

```
# Define function manage_employees that takes parameter employee_updates  
# Initialize department_records as an empty dictionary  
# for department, employee_name, salary in employee_updates:  
# Normalize department and employee names to lowercase for case insensitivity  
# Set normalized department to the lowercase version of department
```

